CS110 / INDEPENDENCE ROAD

Working draft not an issued form / Lexxie

Issued: Nov 10, 2026, 2:55 PM Pacific

Record: WORKING-DRAFT

Personal public verification code: VIEW-9265F91D9B43EF8355AB19A9

Include your own public code with every assignment, even in a team folder. Never submit your private recovery key.

SHA-256: Not issued. No authentication number. Create the numbered forms in the assignment.

Verify the record number at courses.addp2p.com/development-road/independence/index.html?view=verify

An issued record preserves submitted information. It is not a grade or proof of comprehension.

prep

repo

https://github.com/Lexicon-Lexxie/CS110-Lexxie

gitTest

I made my folder inside my repository, with the structure with 12A, I added a plain txt file just so there is something in there for now.

access

the git is public, I plan on doing this project solo this time around.

video1

Observation:

I reviewed the pipeline workflow video and observed the importance of establishing a standardized project folder hierarchy and living documentation before writing any code or importing random assets.

Useful Idea & Interpretation:

A core idea is that a solid, predetermined folder structure combined with modular ReadMe/HTML files isn't just file management it is a team communication tool. When every team member knows exactly where models, scripts, audio, and documentation belong, onboarding is seamless and the risk of merge conflicts or misplaced dependencies is greatly reduced.

Question:

What naming conventions and folder templates does our team plan to agree upon so we avoid broken references when sharing assets in Unity/version control?

video2

Observation:

I reviewed the developer mindset video and observed that technical setbacks and bugs are framed not as personal shortcomings, but as normal, necessary milestones in the development cycle.

Useful Idea & Interpretation:

A core idea presented is treating failure as data rather than defeat. In software and game development, builds break and features fail routinely. Having a growth mindset means actively dissecting mistakes, embracing challenges outside our current comfort zone, and focusing on effort rather than fixed ability so that roadblocks turn into learning opportunities.

CS110 / INDEPENDENCE ROAD

Question:

When we run into code or design failures during sprints, what process will we use as a team to debrief and troubleshoot without placing blame?

video3

Observation:

I reviewed the folder and project setup demonstration and observed how establishing a dedicated directory structure (such as separating Context, Requirements, Resources, and dedicated game build folders) prevents clutter before any engine work begins.

Useful Idea & Interpretation:

A critical idea is structuring folders not just for personal convenience, but for team accessibility and onboarding. By utilizing JSON templates or standardized folder blueprints that include local ReadMe files and requirement checklists, any new team member can immediately understand where assets live, what each folder is for, and what prerequisites are needed without constant back-and-forth communication.

Question:

What core folder structure and root path naming rules will our team enforce before creating the shared Unity repository to ensure everyone stays consistent?

video4

Observation:

I reviewed the documentation video and observed that a Game Development Document (GDD) is treated not as a static, one-time chore, but as a living blueprint that evolves alongside the game as mechanics and milestones are tested.

Useful Idea & Interpretation:

A central idea is that documentation acts as the single source of truth for the entire team defining the narrative, core game loop, required art/audio assets, and individual member responsibilities. My interpretation is that having clear, centralized documentation prevents disputes and scope creep because every decision is measured against the agreed-upon design pillars rather than vague assumptions.

Question:

Who on our team will be responsible for keeping the GDD updated as we iterate, and where will it be hosted so everyone has real-time access?

video5

Observation:

I reviewed the introduction to Unity video and observed the core layout of the Unity Editor specifically the relationship between the Hierarchy (objects currently in the scene), Scene view (3D editor viewport), Game view (rendered player camera), Inspector (object properties and components), and the Project window (asset file structure).

Useful Idea & Interpretation:

CS110 / INDEPENDENCE ROAD

Mastering viewport navigation shortcuts (holding Alt + Left Click to orbit, Middle Click to pan, and pressing F to focus/frame a selected object) along with Ctrl + D to duplicate assets is essential for developer velocity. Being comfortable navigating 3D space and quickly switching between Scene and Game view prevents disorientation and allows team members to build and iterate on environments much faster.

Question:

What default editor layout and common tool settings should our team standardize so our screens and workflows look consistent during screen shares or pairing sessions?

video6

Observation:

I reviewed the custom player import video and observed how a 3D model (like an FBX downloaded from Mixamo) can immediately inherit existing controller logic and animations by setting its Rig Animation Type to Humanoid and assigning its new Avatar to the player's Animator component.

Useful Idea & Interpretation:

Unity s Humanoid rig system acts as a universal bone-mapping bridge. As long as our imported custom character and the template controller are both rigged as Humanoid avatars, we don't have to rebuild running, jumping, or sprint animations from scratch. This allows our art team to swap character models effortlessly while keeping core gameplay mechanics intact.

Question:

What character scale and bone naming conventions will our artists follow in Mixamo/Blender to ensure custom models retarget cleanly to Unity s Humanoid avatar system?

opinion

Observation:

Connecting our pipeline videos to my concept for StimuLye (where the protagonist "NaN" reconstructs a hospital and liminal memory rooms from total darkness), I decided that our team must strictly limit our initial build to the "Vertical Slice" demonstrated in the pipeline focusing only on the dark opening, one looping ward corridor, and one interactive memory room.

Useful Idea & Interpretation:

The pipeline's emphasis on modular Prefabs and scope management is the exact vehicle needed to make StimuLye playable. In the story, interacting with "Stimuli" objects restores stability to the dark void. By building our hospital walls, doors, and memory props as modular Unity prefabs with shared materials, we can dynamically spawn or trigger room transformations without having to hand-model endless unique rooms. Furthermore, formalizing this in our GDD keeps us from expanding into all four acts before the core sensory-restoration mechanic actually feels compelling.

Question:

How can we design our modular corridor and room prefabs so that a simple material or lighting swap can visually transition the hospital from normal to decaying liminal space as "Static" approaches?

questions

What I Did / Decided / Observed:

CS110 / INDEPENDENCE ROAD

I reviewed the complete preparation pipeline from developer mindset and folder architecture to GDD documentation, prefabs, and character rigging. Based on our past discussions around StimuLye, I decided that before committing to a final team role and project mission, we must resolve our core technical constraints, role boundaries, and MVP scope so we don't encounter friction or scope creep mid-sprint.

Questions to Resolve Before Committing:

Scope & MVP: Are we all in agreement to build strictly a tight vertical slice (the dark opening, one looping ward corridor, and one interactive memory room) rather than attempting all four acts right away?

Technical Pipeline: What specific Unity version (e.g., 2021.3 LTS), render pipeline (URP), and version control system (GitHub / Unity Version Control) are we locking in to avoid broken package dependencies?

Role Ownership: Who is taking primary ownership as Lead vs. Support across the essential pipeline roles (Programming/Player Controller, 3D Assets/Modular Prefabs, Sound Design, and GDD Documentation)?

currentPage

44

Whiteboard

How the Pipeline Connects from Start to Finish:

Mindset Sets the Foundation:

Game development projects don't usually fail due to lack of imagination; they fail from unmanaged scope creep, burnout, and uncoordinated communication. The pipeline starts with a developer mindset: committing to small, achievable milestones, viewing bugs and failed builds as iteration data rather than failure, and prioritizing a playable Minimum Viable Product (MVP) over trying to build a 4-act game all at once.

Documentation Turns Ambition into Clear Boundaries:

The Game Development Document (GDD) translates creative concepts (like StimuLye's coma world and "Static" entity) into specific technical requirements. Instead of an unread 100-page document, it functions as a visual, living blueprint containing our core loop, milestone checklists, and explicit role definitions (Lead vs. Support) so everyone knows what to build without ambiguity.

Folder Architecture Protects the Project:

Before launching the engine, creating a clean directory structure with shallow file paths prevents technical issues like the Windows 256-character decompression limit. Placing change logs (.LOG) and local ReadMes inside dedicated folders (/Requirements, /Resources, /Context) ensures that code, art, and references are organized before team members start merging files.

Unity Implementation via Templates & Prefabs:

Inside Unity, the pipeline emphasizes efficiency over reinventing the wheel:

CS110 / INDEPENDENCE ROAD

Starting from a verified Third Person Template provides instant player movement, physics, and camera controls.

Keeping custom assets in a dedicated team subfolder (/Assets/MyGame/) keeps third-party plugins and asset packages from cluttering our work.

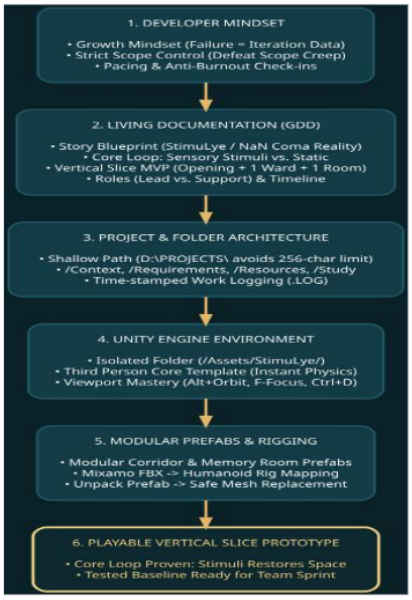
Using Prefabs (the "cookie-cutter" model) allows environment pieces and interactive props to be duplicated and updated across scenes simultaneously without wasting memory.

Safe Character & Mechanic Integration:

To bring our custom story to life, the pipeline uses Unity's Humanoid Avatar retargeting. By unpacking the template armature, we can safely remove the placeholder mannequin and insert our custom character mesh without breaking underlying movement logic.

Conclusion:

Every phase feeds directly into the next: Mindset keeps the project scoped, Documentation defines the mission, Folder Structure prevents file corruption, and Prefabs/Retargeting allow us to build a playable, atmospheric prototype quickly and sustainably.



pitch

understand

What I Did / Decided / Observed:

Moving from the preparation phase into active contribution, I observed that our objective is not to jump blindly into coding a full game, but to execute a standardized development pipeline. I decided that our team's immediate mission must focus on delivering a cohesive, playable vertical slice rather than getting bogged down in feature bloat.

CS110 / INDEPENDENCE ROAD

The Work (What Needs to Be Done):

"The work" means setting up a structured foundation before building out content. This involves:

Establishing a clean, shallow directory structure (/Context, /Requirements, /Resources, /Assets/StimuLye/) and initializing our living GDD with clear Lead/Support role assignments.

Setting up the Unity project with a verified Third-Person template to secure immediate movement physics and camera controls.

Importing and retargeting our custom 3D model (swapping the default mannequin with NaN s avatar via Unity's Humanoid rig system).

Building modular corridor and room prefabs to establish the environment.

The Finish Line (Definition of Done):

The finish line is not a finished four-act game; it is a playable Vertical Slice prototype (MVP). Completion means having a stable, testable build where a player wakes up in the dark, navigates the opening ward corridor, and successfully interacts with a "Stimuli" item that visibly stabilizes a memory room without errors or broken dependencies.

Question / Next Step:

Once we verify that the player can walk through the corridor and trigger the first room transition, what specific performance and gameplay metrics will we test during our huddle?

offer

I feel that I am a good leader who brings good structure to the team to maximize the efficiency of a team. I also like to see myself as a jack of all trades as I am able to adapt to new environments to provide good quality work to any field.

limits

I feel like I need to manage my time a bit better since I have been becoming very overwhelmed with the team projects and their deadlines, mainly because people aren't as "reliable" as I would hope.

availability

2026-11-10 16:00 17:00 Pacific

2026-11-12 16:00 17:00 Pacific

Preferred contact: Discord

My Discord (Samdan9) is pretty much always available.

tools

I would be using my IDE, Unity engine, and I would also use AI for some of the planning stuff.

aiTool

AI: Claude

Task: Help me with the heavy lifting, which would be some of the code writing and organization.

Verification: I will ensure that it works and does not infringe on any copyright to avoid conflicts.

availability

CS110 / INDEPENDENCE ROAD

notes

My Discord (Samdan9) is pretty much always available.

contact

Discord

slots

0

id

8837512c-cbc9-4881-8d21-b3bd15883880

date

2026-11-10

Class starts

16:00

end

17:00

1

id

5f3edf29-d83d-4736-92c6-ca4055e59574

date

2026-11-12

Class starts

16:00

end

17:00

aiPlan

Name

Claude

task

Help me with the heavy lifting, which would be some of the code writing and organization.

verify

I will ensure that it works and does not infringe on any copyright to avoid conflicts.

huddle

mission

I want to make a beautiful playable scene in unity with a 3D player viewed in 3rd person.

framework

I will use the AAA framework in order to keep an organized structure of how I am able to complete my project efficiently.

pipeline

Unnamed stage

Unnamed stage

CS110 / INDEPENDENCE ROAD

I used mermaid owner Lexxie

Mermaid worked better for me.

Production Pipeline Explanation: StimuLye Vertical Slice (AAA Framework)

Our production pipeline applies Dr. Malik Stalbert's AAA Framework (Application Action Amplification) to transition our team smoothly from an initial template into a playable, narrative-driven prototype:

1. Stage 1: Application (Setup & Baseline)

Following the principle that "application gets them moving," we do not start by writing code from scratch. We establish our shallow directory structure (D:\PROJECTS\StimuLye) to bypass Windows path length limits and deploy Unity's verified Third Person Core template. This gives us immediate physics, camera orbit, and movement controls within 30 minutes.

2. Stages 2 & 3: Action (Mechanics, Avatars & Environments)

Once the baseline is alive, the team enters the "Action" phase, where concepts and custom assets are introduced:

- Character Integration: We import our NaN model, set its rig to Humanoid, safely unpack the player armature, and swap the geometry. The Humanoid avatar acts as the bridge so existing template animations retarget cleanly to our custom protagonist.
- Environment Construction: Our level designer uses modular prefabs (the "cookie-cutter" approach) to build the looping hospital ward and dark memory rooms without wasting memory.

3. Stage 4 & 5: Amplification (Core Loop, Polish & Verification)

In the "Amplification" phase, private technical competence expands into an integrated experience:

- Core Narrative Mechanic: We implement the "Stimuli" interaction script. Picking up the sensory object triggers a room material swap from dark void to stable hospital textures while playing the first "Static" audio cue.
- Verification & Team Review: The team conducts an end-to-end playtest, verifying that the opening dark sequence, corridor movement, and memory stabilization run smoothly with zero console errors.

Roles & Time Estimates:

- Block 1 (30m): Technical Lead (Template & Directories)
- Block 2 (45m): 3D Artist / Character Lead (Rigging & Retargeting)
- Block 3 (60m): Level Designer (Modular Ward Prefabs)
- Block 4 (60m): Gameplay Programmer & Audio Lead (Stimuli Triggers)
- Block 5 (45m): QA & Entire Team (Playtest & Milestone Sign-off)

Total Sprint Duration: 4 Hours (240 Minutes)

tech

What I Did / Decided / Observed:

As a solo developer on StimuLye, my priority is eliminating pipeline friction so I don't get overwhelmed wearing every hat alone. I applied the TnT ("Tool and the Temple") principle: every tool must serve my MVP vertical slice and reduce manual labor. I chose a lean, verified stack built on Unity 6.6 and established personal testing gates to check my own work at each milestone.

CS110 / INDEPENDENCE ROAD

My Agreed Solo Tool Stack & Verification Plan:

Engine: Unity 6.6 (6000.6) with URP

Why: Faster Play Mode entry, lighter memory footprint, and native Universal Render Pipeline lighting/fog to achieve StimuLye s dark atmosphere without third-party render plugins.

Who Checks It: Me (wearing the Technical Hat) verified via build tests and frame rate checks.

Language & IDE: C# / Visual Studio Community

Why: Standard Unity scripting for clean, fast implementation of the "Stimuli" interaction and room material swap trigger scripts.

Who Checks It: Me (wearing the Programmer Hat) verified with zero console errors and clean syntax.

Base Template: Starter Assets Third Person Controller + Cinemachine

Why: Provides an instant movement, sprint, jump, and camera follow baseline. As a solo dev, this saves me dozens of hours of physics and camera programming so I can focus directly on the story mechanics.

Who Checks It: Me (wearing the Technical Hat) verified through Play Mode controller feel.

Art & Rigging: Mixamo + Blender

Why: Mixamo automatically generates the Humanoid rig and Avatar for my custom "NaN" character; Blender is used for quick adjustments and whiteboxing modular ward prefabs.

Who Checks It: Me (wearing the Visual Hat) verified by checking that animations retarget without bone stretching.

Audio: Audacity / FreeSound (WAV stems)

Why: Simple, lightweight sound editing for hospital room tone, heart monitor beeps, and the "Static" cue.

Who Checks It: Me (wearing the Audio Hat) verified with in-engine volume balance checks.

Progress Tracking: Git + Project_LOG.txt

Why: Local Git commits protect my project from corruption; the auto-timestamped .LOG file holds me accountable to

CS110 / INDEPENDENCE ROAD

daily tasks.

Who Checks It: Me (wearing the QA / Architect Hat) audited against my GDD checklist.

Question / Self-Check:

How can I automate my daily .LOG entries and Git commits so project tracking remains effortless while working solo?

Time code

Classes & components

Class | Responsibility | GameObject

ThirdPersonController | Processes movement calculations, sprint, jump physics, gravity, and drives animator parameters. | PlayerArmature

StarterAssetsInputs | Reads raw input data (WASD, mouse delta, space, shift) from the Input System and stores state flags. | PlayerArmature

CharacterController | Unity engine physics component that manages capsule collision, ground detection, and slopes. | PlayerArmature

StimulusItem | Detects player proximity, triggers interaction events, and signals memory recovery when collected. | Stimulus_MemoryObject

RoomStabilizer | Listens for Stimulus collection; swaps room shaders/materials from dark void to hospital textures. | Environment_Manager

StaticController | Monitors player distance to boundaries; increases audio distortion and visual grain as "Static" stalks. | Static_Antagonist

PauseMenuController | Listens for the Esc key; pauses time and displays the CRT power-down / heart monitor scene. | UI_Canvas_Manager

Variables / properties

Name : type | Value / purpose | Used by

MoveSpeed | Defines base walk velocity vs. sprint speed in the ward. | ThirdPersonController

_animIDSpeed | Cached hash IDs for fast Animator parameter updates. | ThirdPersonController

isStabilized | Prevents the room transformation from firing more than once. | RoomStabilizer

darkMaterial | References to the low-poly void shader vs. lit hospital texture. | RoomStabilizer

staticProximityThreshold | Distance from player before static distortion audio begins. | StaticController

stimulusSound | Audio stems played on memory pickup and enemy approach. | AudioManager

f(x) Functions

Function | Inputs result | When called

Awake() | Engine Initialization | Caches component references (GetComponent<Animator>(), CharacterController) before scripts run.

Start() | First Frame | Sets the opening scene state: turns off ward lights, applies black void materials, locks cursor.

Update() | Every Frame | Polls player movement vectors, updates camera angles, and checks for the Escape key input.

OnTriggerEnter(Collider) | Physics Collision Event | Detects when NaN steps into the StimulusItem radius or enters a

"Static" trigger volume.

OnFootstep(AnimationEvent) | Animation Event | Plays footstep sound cues synchronized with the walk/run animation clip frames.

Packages / using

Package / namespace | Why included | Tested

using UnityEngine; | The fundamental foundation of all Unity code. Provides access to MonoBehaviour, GameObject, Transform, Vector3, Time, and core physics engine types. | [x]

using UnityEngine.InputSystem; | Replaces the legacy Input.GetKey system. Required in Unity 6 for cross-platform event-based input handling (keyboard, mouse look, and gamepads) with zero frame-rate polling lag. | [x]

using Cinemachine; | Controls the dynamic third-person follow camera rig. Automatically handles procedural damping, collision detection with ward walls, and smooth cursor orbital movement. | [x]

using UnityEngine.Events; | Allows decoupled communication via UnityEvent or Action. When NaN collects a Stimulus, the item simply raises an event; the RoomStabilizer and AudioSource respond without being hardcoded to each other. | [x]

using UnityEngine.Rendering.Universal; | Gives C# runtime control over Unity 6's Universal Render Pipeline (URP) volume components (vignette, film grain, chromatic aberration) to create the visual "Static" distortion effect. | [x]

communication

Team check-ins

Description / contact / owner | When Pacific | Agreed

Lexxie | | [x]

reuse

What I Did / Decided / Observed:

Thinking beyond this initial prototype assignment, I audited our production assets to determine what foundational systems can be reused across Act 2 (The Ward), Act 3 (The Deep), and the Finale without rebuilding from scratch. I decided to maintain a strict licensing and attribution log in /Requirements/ so all third-party credit requirements are tracked before scaling the game.

Reusable Components, Sources & Credit Requirements:

Core Character Controller & Camera Rig:

What is Reusable: The PlayerArmature with Third-Person movement physics, jump/sprint logic, and Cinemachine orbital follow camera.

Source: Unity Technologies (Starter Assets Third Person Character Controller).

Credit / License: Unity Companion License / Asset Store Terms. Free for commercial and non-commercial compiled builds; requires standard Unity engine inclusion in game credits.

Protagonist Model & Rigging:

CS110 / INDEPENDENCE ROAD

What is Reusable: The Humanoid bone hierarchy, animation controller mapping, and custom "NaN" mesh.

Source: Adobe Mixamo (Erika_Archer base mesh & standard Humanoid locomotion clips).

Credit / License: Royalty-free under Adobe Mixamo Terms of Service for use in embedded video game builds (attribution to Adobe Mixamo in credits; raw .fbx files cannot be redistributed open-source).

Modular Room & Corridor Prefab Kit:

What is Reusable: Floor, wall, and doorway prefabs that can be re-skinned into decaying Backrooms/liminal spaces for Act 3 using shader/material swaps.

Source: Custom-modeled in Blender / Unity primitives.

Credit / License: Original creator IP 100% owned, no third-party attribution required.

"Stimuli" Trigger & Room Stabilization Scripts:

What is Reusable: The event-driven C# interaction architecture (StimulusItem.cs, RoomStabilizer.cs) can power every memory puzzle across all remaining chapters.

Source: Custom original C# codebase.

Credit / License: Original creator IP fully reusable across future episodes or spinoff projects.

Audio & Distortion Cues:

What is Reusable: Heartbeat pulses, hospital ambient drone, and the "Static" entity audio stems.

Source: Pixabay / FreeSound.org.

Credit / License: Pixabay Content License / Creative Commons (CC-BY 4.0 / CC0). Requires listing contributing sound artist handles in the game's final end-credits roll.

Question / Next Step:
Should I set up an automated Credits.txt file inside /Requirements/ now so that attribution for new sound stems and models is recorded the moment they are downloaded?

boardNotes

I will continue to work on the small project until I am ready to move up to the team project.

Whiteboard

Production Pipeline Explanation: StimuLye Vertical Slice (AAA Framework)

Our production pipeline applies Dr. Malik Stalbert's AAA Framework (Application Action Amplification) to transition our team smoothly from an initial template into a playable, narrative-driven prototype:

1. Stage 1: Application (Setup & Baseline)

Following the principle that "application gets them moving," we do not start by writing code from scratch. We establish our shallow directory structure (D:\PROJECTS\StimuLye) to bypass Windows path length limits and deploy Unity's verified Third Person Core template. This gives us immediate physics, camera orbit, and movement controls within 30 minutes.

2. Stages 2 & 3: Action (Mechanics, Avatars & Environments)

Once the baseline is alive, the team enters the "Action" phase, where concepts and custom assets are introduced:

- Character Integration: We import our NaN model, set its rig to Humanoid, safely unpack the player armature, and swap the geometry. The Humanoid avatar acts as the bridge so existing template animations retarget cleanly to our custom protagonist.
- Environment Construction: Our level designer uses modular prefabs (the "cookie-cutter" approach) to build the looping hospital ward and dark memory rooms without wasting memory.

3. Stage 4 & 5: Amplification (Core Loop, Polish & Verification)

In the "Amplification" phase, private technical competence expands into an integrated experience:

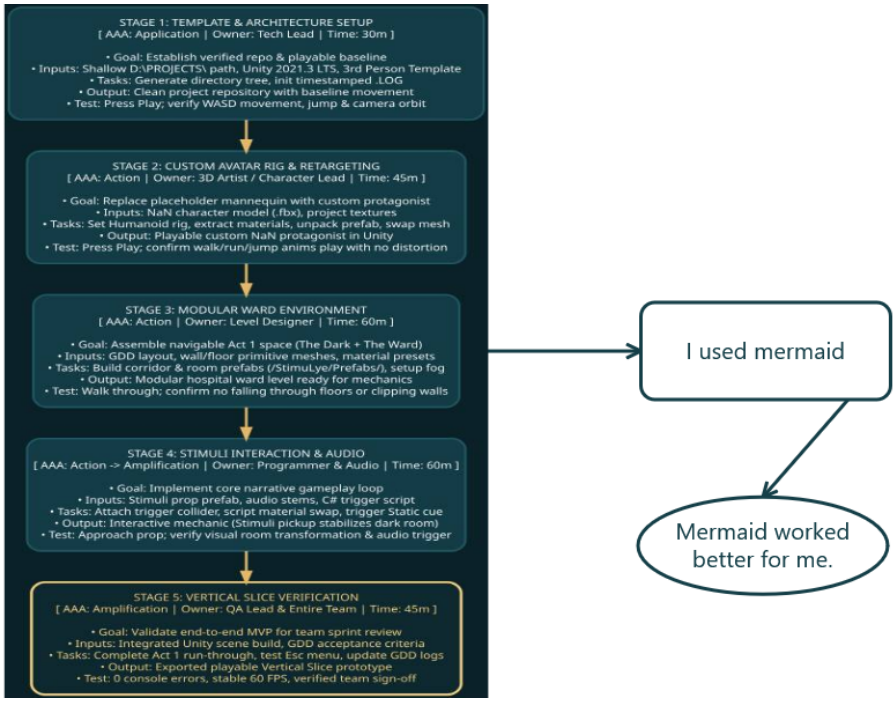
- Core Narrative Mechanic: We implement the "Stimuli" interaction script. Picking up the sensory object triggers a room material swap from dark void to stable hospital textures while playing the first "Static" audio cue.
- Verification & Team Review: The team conducts an end-to-end playtest, verifying that the opening dark sequence, corridor movement, and memory stabilization run smoothly with zero console errors.

Roles & Time Estimates:

- Block 1 (30m): Technical Lead (Template & Directories)
- Block 2 (45m): 3D Artist / Character Lead (Rigging & Retargeting)
- Block 3 (60m): Level Designer (Modular Ward Prefabs)
- Block 4 (60m): Gameplay Programmer & Audio Lead (Stimuli Triggers)
- Block 5 (45m): QA & Entire Team (Playtest & Milestone Sign-off)

Total Sprint Duration: 4 Hours (240 Minutes)

CS110 / INDEPENDENCE ROAD



codeTables

0

id

061b1987-dcfd-45cf-80bc-0948d0a0ace7

title

Classes & components

columns

0

label

Class

type

text

1

label

Responsibility

type

text

2

label

GameObject

type

text

rows

CS110 / INDEPENDENCE ROAD

0

0

ThirdPersonController

1

Processes movement calculations, sprint, jump physics, gravity, and drives animator parameters.

2

PlayerArmature

1

0

StarterAssetsInputs

1

Reads raw input data (WASD, mouse delta, space, shift) from the Input System and stores state flags.

2

PlayerArmature

2

0

CharacterController

1

Unity engine physics component that manages capsule collision, ground detection, and slopes.

2

PlayerArmature

3

0

StimulusItem

1

Detects player proximity, triggers interaction events, and signals memory recovery when collected.

2

Stimulus_MemoryObject

4

0

RoomStabilizer

1

Listens for Stimulus collection; swaps room shaders/materials from dark void to hospital textures.

2

Environment_Manager

5

0

StaticController

1

CS110 / INDEPENDENCE ROAD

Monitors player distance to boundaries; increases audio distortion and visual grain as "Static" stalks.

2

Static_Antagonist

6

0

PauseMenuController

1

Listens for the Esc key; pauses time and displays the CRT power-down / heart monitor scene.

2

UI_Canvas_Manager

1

id

6a62977a-857f-4e05-b30d-ec5a07798942

title

Variables / properties

columns

0

label

Name : type

type

text

1

label

Value / purpose

type

text

2

label

Used by

type

text

rows

0

0

MoveSpeed

1

Defines base walk velocity vs. sprint speed in the ward.

2

ThirdPersonController

CS110 / INDEPENDENCE ROAD

1

0

_animIDSpeed

1

Cached hash IDs for fast Animator parameter updates.

2

ThirdPersonController

2

0

isStabilized

1

Prevents the room transformation from firing more than once.

2

RoomStabilizer

3

0

darkMaterial

1

References to the low-poly void shader vs. lit hospital texture.

2

RoomStabilizer

4

0

staticProximityThreshold

1

Distance from player before static distortion audio begins.

2

StaticController

5

0

stimulusSound

1

Audio stems played on memory pickup and enemy approach.

2

AudioManager

2

id

595bbdd0-1eb9-4603-a8b8-c8d31f390b23

title

CS110 / INDEPENDENCE ROAD

f(x) Functions

columns

0

label

Function

type

text

1

label

Inputs result

type

text

2

label

When called

type

text

rows

0

0

Awake()

1

Engine Initialization

2

Caches component references (GetComponent<Animator>(), CharacterController) before scripts run.

1

0

Start()

1

First Frame

2

Sets the opening scene state: turns off ward lights, applies black void materials, locks cursor.

2

0

Update()

1

Every Frame

2

Polls player movement vectors, updates camera angles, and checks for the Escape key input.

CS110 / INDEPENDENCE ROAD

3

0

OnTriggerEnter(Collider)

1

Physics Collision Event

2

Detects when NaN steps into the StimulusItem radius or enters a "Static" trigger volume.

4

0

OnFootstep(AnimationEvent)

1

Animation Event

2

Plays footstep sound cues synchronized with the walk/run animation clip frames.

3

id

b0f46e68-aa40-4beb-a479-052e7cd5204d

title

Packages / using

columns

0

label

Package / namespace

type

text

1

label

Why included

type

text

2

label

Tested

type

checkbox

rows

0

0

using UnityEngine;

CS110 / INDEPENDENCE ROAD

1

The fundamental foundation of all Unity code. Provides access to MonoBehaviour, GameObject, Transform, Vector3, Time, and core physics engine types.

2

true

1

0

using UnityEngine.InputSystem;

1

Replaces the legacy Input.GetKey system. Required in Unity 6 for cross-platform event-based input handling (keyboard, mouse look, and gamepads) with zero frame-rate polling lag.

2

true

2

0

using Cinemachine;

1

Controls the dynamic third-person follow camera rig. Automatically handles procedural damping, collision detection with ward walls, and smooth cursor orbital movement.

2

true

3

0

using UnityEngine.Events;

1

Allows decoupled communication via UnityEvent or Action. When NaN collects a Stimulus, the item simply raises an event; the RoomStabilizer and AudioSource respond without being hardcoded to each other.

2

true

4

0

using UnityEngine.Rendering.Universal;

1

Gives C# runtime control over Unity 6's Universal Render Pipeline (URP) volume components (vignette, film grain, chromatic aberration) to create the visual "Static" distortion effect.

2

true

communicationTables

0

id

CS110 / INDEPENDENCE ROAD

5a24f4fd-3441-4b83-8102-50851844f53a

title

Team check-ins

columns

0

label

Description / contact / owner

type

text

1

label

When Pacific

type

datetime-local

2

label

Agreed

type

checkbox

rows

0

0

Lexxie

1

2

true

Task list

0

id

75169224-740e-41a8-8b8a-24a6137bee77

task

Documentation

Task owner reference

PERSON-534364FED196749589B55B74

Deadline

2026-11-12T20:30

Completed

false

CS110 / INDEPENDENCE ROAD

Execution / result

1

id

de1342be-2e15-45b8-a532-e54167c65a25

task

Setup

Task owner reference

PERSON-534364FED196749589B55B74

Deadline

2026-11-12T20:30

Completed

false

Execution / result

2

id

ea92e30c-008a-4533-9ce6-e2786e733ed4

task

Player import

Task owner reference

PERSON-534364FED196749589B55B74

Deadline

2026-11-12T20:30

Completed

false

Execution / result

3

id

3ba2fd21-ccbe-4d2d-8b62-3282e36cbb0b

task

Testing

Task owner reference

PERSON-534364FED196749589B55B74

Deadline

2026-11-12T20:30

Completed

false

CS110 / INDEPENDENCE ROAD

Execution / result

4

id

66088a36-ec28-4a94-a6d7-d33cb121cf18

task

Handoff

Task owner reference

PERSON-534364FED196749589B55B74

Deadline

2026-11-12T20:30

Completed

false

Execution / result

Whiteboard

Diagram: StimuLye vertical slice plan (solo, Unity 6.6, AAA framework)

The mission is a playable vertical slice: a dark opening, one looping ward corridor, and one Stimuli memory room. I'm working solo, so each box shows the "hat" I wear.

Stage 1 (Application, Technical hat): set up the project with Unity 6.6 + URP and the Third Person Starter Assets so I have working movement immediately. I also create the folders and Project_LOG.txt. Test: WASD, jump, and camera work.

Stage 2 (Action, 3D Artist hat): import the Mixamo NaN model, set it to Humanoid, unpack PlayerArmature, and swap the geometry. Test: it runs and jumps with no pink shaders.

Stage 3 (Action, Level Designer hat): build modular wall, floor, and door prefabs with colliders and dark fog. Test: no clipping or falling through floors.

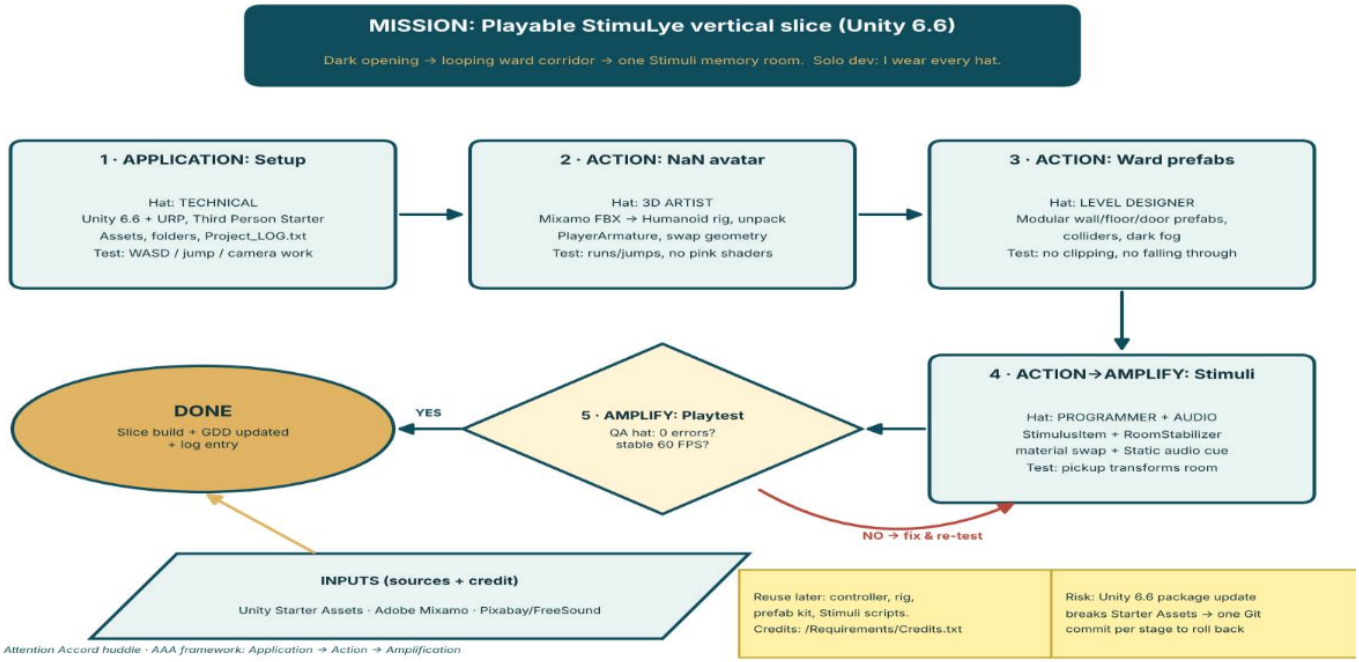
Stage 4 (Action to Amplification, Programmer + Audio hat): StimulusItem triggers RoomStabilizer, which swaps dark materials for hospital ones and plays the Static audio cue. Test: pickup visibly transforms the room.

Stage 5 (Amplification, QA hat): the decision diamond. If the playtest shows errors or low FPS, I loop back to stage 4 and fix. If it is clean (0 errors, stable 60 FPS), the slice is Done and I update the GDD and log.

Inputs are Unity Starter Assets, Adobe Mixamo, and Pixabay/FreeSound. Every credit goes in /Requirements/Credits.txt so these parts can be reused in later acts.

Risk: a Unity 6.6 package update could break Starter Assets, so I make one Git commit per stage to roll back.

CS110 / INDEPENDENCE ROAD



planRevision

1

reportA

checkpoint

I finished the entirety of week 12a which was a lot for me, but it was worth it.

evidence

https://github.com/Lexicon-Lexxie/CS110-Lexxie/tree/main/Week12/12A

Every file is in the 12a folder :)

next

My next plan is to work on the unity engine

Whiteboard

Checkpoint: StimuLye vertical slice (solo, Unity 6.6)

This board is my checkpoint for today's huddle. I'm working alone, so it records where my plan stands and what I do next.

Decided today: I limited the project to one vertical slice (the dark opening, one looping ward corridor, and one Stimuli memory room) instead of the full four-act game. I chose Unity 6.6 with URP, C#, the Third Person Starter Assets, Mixamo for the NaN character, and Git for version control. I organized the work into five stages following the AAA framework: Setup, Avatar, Prefabs, Stimuli mechanic, and Playtest.

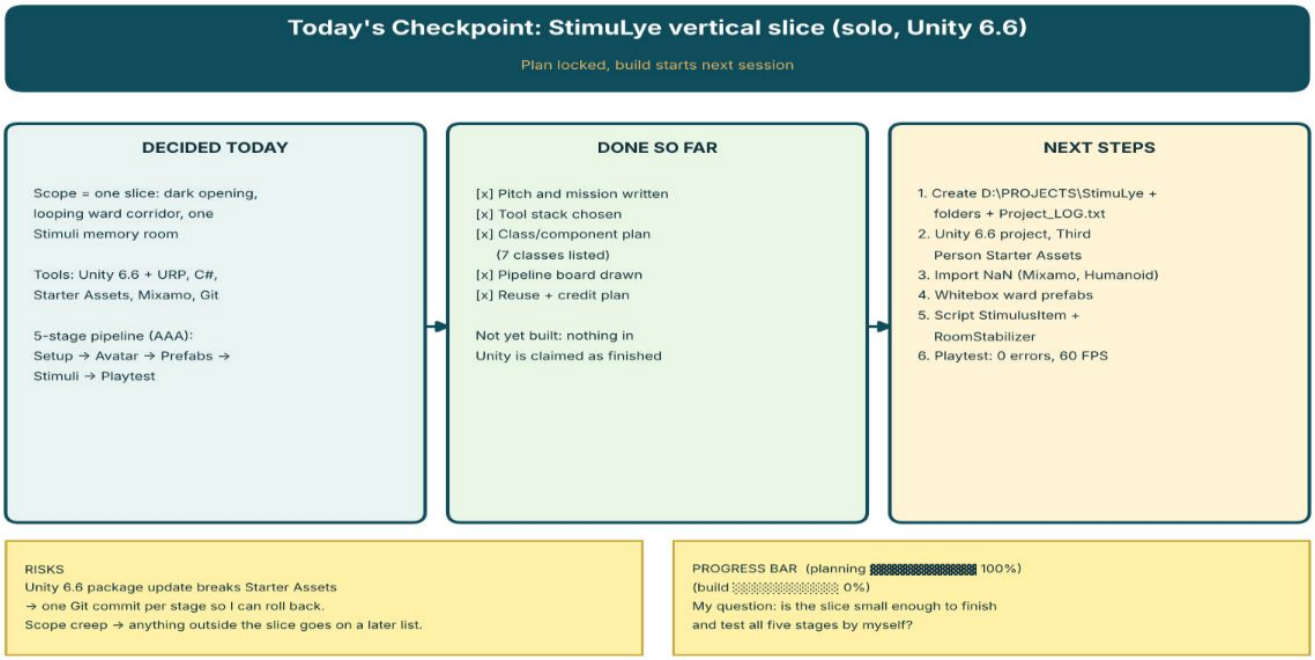
Done so far: all the planning is finished. That means the pitch and mission, the tool stack, the class and component plan, the pipeline board, and the plan for reusing assets and crediting their sources. I have not built anything in Unity

yet, so the build is at 0% and the planning is at 100%.

Next steps: I will create the project folders and Project_LOG.txt, then start the Unity project from the Third Person template. After that I will import the NaN character, whitebox the ward prefabs, write the StimulusItem and RoomStabilizer scripts, and finish with a playtest. The target is no console errors and a stable 60 FPS.

Risks: a Unity 6.6 package update could break the Starter Assets, so I will make one Git commit per stage so I can roll back. Anything outside the slice goes on a later list to prevent scope creep.

Question for feedback: is this slice small enough for me to finish and test all five stages by myself?



Name

Lexxie